STRATEGY | Estratégia

Um sistema de pagamentos com comportamentos intercambiáveis

Apresentação acadêmica • Exemplo prático em PHP

O que são padrões de projeto?

- Soluções reutilizáveis para problemas recorrentes no desenvolvimento de software.
- Não são códigos prontos: são modelos de organização e colaboração entre classes.
- O Strategy pertence à categoria de padrões comportamentais.

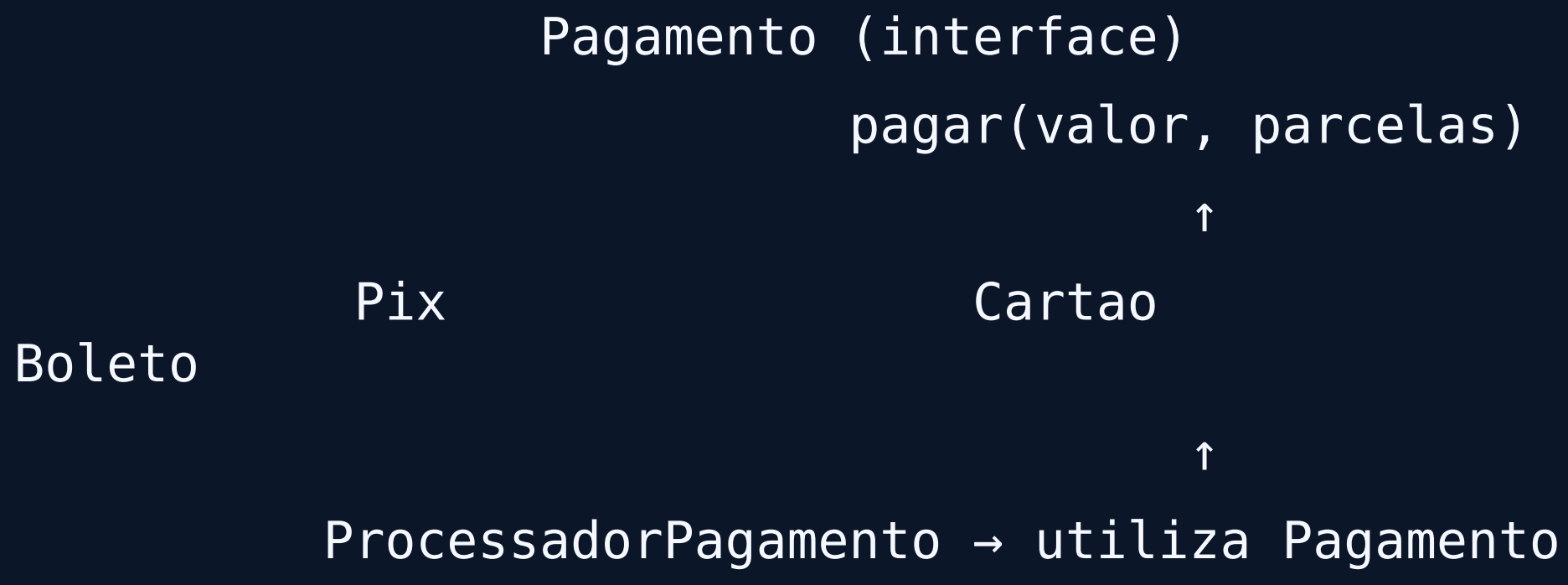
O que é Strategy?

- Define uma família de algoritmos, encapsula cada um e permite trocá-los.
- O código cliente utiliza uma interface comum, sem depender dos detalhes de cada algoritmo.
- No nosso exemplo, cada forma de pagamento é uma estratégia.

Qual problema ele resolve?

- Sem Strategy: regras de PIX, cartão e boleto misturadas em uma única classe.
- Adicionar ou alterar pagamentos pode aumentar condicionais e acoplamento.
- Com Strategy: cada comportamento fica em sua própria classe.

Estrutura do nosso exemplo



A classe processadora trabalha com a interface, não com uma forma específica.

Interface: contrato comum

A interface define um contrato: qualquer forma de pagamento precisa oferecer o método pagar().

```
interface Pagamento {
    public function pagar($valor, $parcelas = 1);
}

// Todas as estratégias implementam pagar().
```

Três estratégias de pagamento

PIX e boleto são à vista; cartão permite de 1 a 6 parcelas. Cada classe implementa a mesma interface.

```
class Pix implements Pagamento {
    public function pagar($valor, $parcelas = 1) {
        return ["forma" => "PIX", "parcelas" => 1];
    }
}

// Cartao: aceita de 1 a 6 parcelas.
// Boleto: pagamento à vista (1 parcela).
```

Processador: ponto central

O processador recebe um objeto Pagamento e chama pagar(), sem depender da classe concreta.

```
class ProcessadorPagamento {
    private Pagamento $pagamento;

    public function __construct(Pagamento $pagamento) {
        $this->pagamento = $pagamento;
    }

    public function processar($valor, $parcelas = 1) {
        return $this->pagamento->pagar($valor, $parcelas);
```

Como a estratégia é escolhida

O formulário informa a escolha; o PHP cria a estratégia adequada e entrega ao processador.

```
switch ($forma) {
    case "pix":    $pagamento = new Pix(); break;
    case "cartao": $pagamento = new Cartao(); break;
    case "boleto": $pagamento = new Boleto(); break;
}

$processador = new ProcessadorPagamento($pagamento);
$resultado = $processador->processar($valor, $parcelas);
```

Demonstração ao vivo

- Abrir http://localhost/strategy_trabalho/index.php no XAMPP.
- Testar PIX: R$ 150,00, pagamento em 1 vez.
- Testar cartão: R$ 300,00 em 3 parcelas de R$ 100,00.
- Testar boleto: R$ 200,00, pagamento em 1 vez.
- Exibir e imprimir o comprovante fictício.

Vantagens e limitações

VANTAGENS

✓ Separação de responsabilidades

✓ Facilidade para adicionar estratégias

✓ Código mais fácil de testar

✓ Troca de comportamento

LIMITAÇÕES

• Mais classes e arquivos

• Exige compreender interfaces

• Pode ser excessivo em casos simples

Conclusão

- O Strategy organiza diferentes comportamentos atrás de uma interface comum.
- O mesmo processador executa PIX, cartão e boleto sem conhecer suas implementações.
- O projeto demonstra o padrão com um sistema funcional e comprovantes simulados.

Referências

REFACTORING.GURU. Strategy. https://refactoring.guru/pt-br/design-patterns/strategy

SOURCEMAKING. Strategy Design Pattern. https://sourcemaking.com/design_patterns/strategy

PHP. Interfaces de objetos. https://www.php.net/manual/pt_BR/language.oop5.interfaces.php

GAMMA, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.